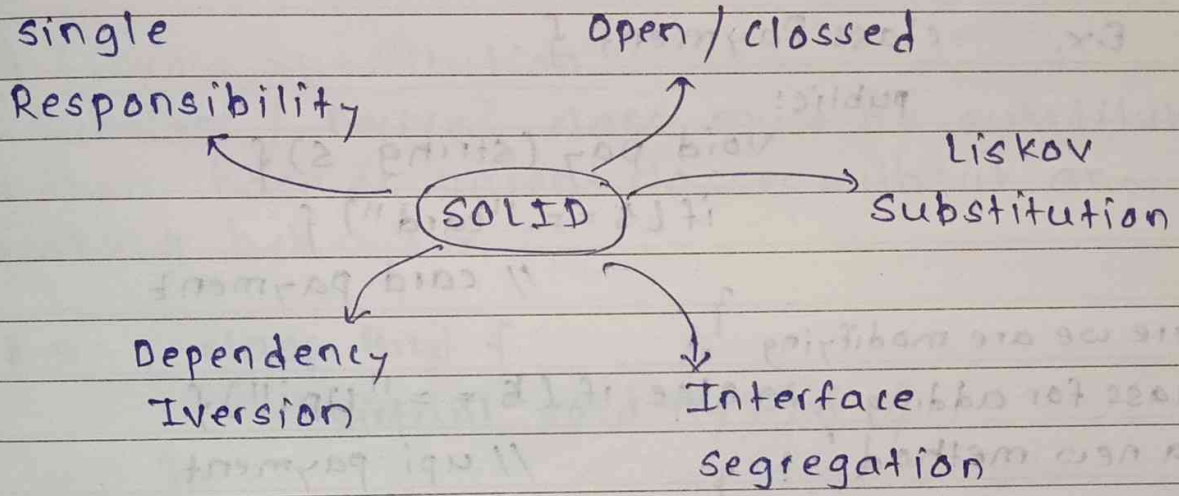
Solid Principles:

June
Wednesday
2024
19

Software development requires careful planning & design to ensure that applications are maintainable, scalable & easy to understand.

One of the foundational guidelines for acheiving these goals is SOLID Principles.



(1) Single Responsibility: A class should have only one reason to change. In other words, a class should have only one job or purpose within the software system.

E.x. A baker is responsible for baking bread. However if the baker is also responsible for:

* Managing inventory
* Ordering supplies
* serving customers
* cleaning the bakery

Each of these tasks represents a separate responsibility & by combining them, the baker's focus & effictiveness in baking bread could be compromised.

Notes

20 June Thursday 2024

JUNE 2024

S	M	T	W	T	F	S
30						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29

172-194 Week 25

(2) Open / closed Principle:

Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification.

We should be able to add new functionality without modifying existing one.

E.x.

```
class Payment {
public:
    void pay(string s) {
        if (s == "card") {
            // card payment
        }
        else if (s == "upi") {
            // upi payment
        }
    }
}
```

Here we are modifying class for adding a new method which can be risky.

Correct way:

```
class Payment {
public:
    virtual void pay() = 0;
};
class Card : public Payment {
    void pay() override {
        // card payment
    }
};
```

Notes

```
class Upi : public Payment {
    void pay() override {
        // upi method
    }
};
```

Adding a new payment method without modifying existing class

---

(3) Liskov Substitution:

Subclass / Derived class must be substitutable for their base or parent classes without any breaking behaviour.

E.x.

```
class Bird {
    virtual void fly() { }
};
class Sparrow : public Bird {
    void fly() override { }
};
class Penguin : public Bird {
    void fly() override {
        throw "Penguins can't fly";
    }
};
```

Here Penguin breaks behaviour

Notes

correct way:

```
class Bird { };
class FlyingBird : public Bird {
        virtual void fly() { }
};
class Sparrow : public FlyingBird {
        void fly() override { }
};
class Penguin : public Bird {
        // no flying method
}
```

---

4) Interface segregation Principle:

clients should not be forced to depend on interfaces they do not use.

```
class Machine {
        virtual void print() = 0;
        virtual void scan() = 0;
};
class Printer : public Machine {
        void print() override { }
        void scan() override { } ==> Not needed for printer but still forced to implement it.
};
```

correct way:

```
class Printer {
        virtual void print() = 0;
};
class scanner {
        virtual void scan() = 0;
};
class AllInOne : public Printer, public Scanner
{                                    };
```

---

* Dependency Inversion Principle

Classes should rely on abstractions rather than concrete implementations.

* High-level modules should not depend on low-level modules.
Both should depend on abstractions.

E.x.

```
class MySQLDatabase {
        void save() { }
};
class OrderService {
        MySQLDatabase db;
        void placeOrder() {
                db.save();
        }
};
```

Tightly coupled

If we have to shift to MongoDB → we must modify OrderService

It will be hard to test.

Notes

JULY

AUGUST

Now instead of depending on concrete class, depend on abstract (interface).

```
class Database {
    virtual void save() = 0;
};
class MySQL : public Database {
    void save() override { }
};
class MongoDB : public Database {
    void save() override { }
};

class OrderService {
    Database *db;
    OrderService(Database* d) {
        db = d
    };
};

MySQL m1;
OrderService o1(&m1);
MongoDB m2;
OrderService o2(&m2);
```

Injecting dependency

Usage

Notes

* DRY (Don't Repeat Yourself): To avoid duplicating code in a system.
Every piece of knowledge or logic should exist in only one place in your code.

* KISS (Keep it simple, stupid):
Always write the simplest solution that works.
Avoid unnecessary complexity.

E.x. Bad (complex):
```
if(user.role == "admin" || user.role == "ADMIN"
|| user.role == "Admin") { }
```
Good (simple):
```
string role = toLowerCase (user.role);
    if (role == 'admin") { }
```

* YAGNI (You aren't gonna need it):
Don't add functionality until it is actually needed.
Write code only for current requirements & not for future assumptions.

E.x.
```
class AuthService {
        void loginWithEmail() { }
Notes not required yet
        void loginWithGoogle() { }
        void loginWithFacebook() { }
        void loginWithOTP() { }
};
```

27 June Thursday 2024

JUNE 2024

S	M	T	W	T	F	S
30						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29

Week 26

179-187

* Creational Design Patterns:

Creational design patterns deal with how objects are created.

They help create objects in a flexible, reusable and controlled way instead of using new everytime.

5 types of CDP:

(1) **Singleton:** Ensures there is only one instance of a class throughout the entire system.

This prevents resource wastage & ensures consistent behaviour.

E.x. A single database connection shared across the whole system.

```
class Database {
    static Database* instance;

    Database() { }

public:
    static Database* getInstance() {
        if (instance == NULL) {
            instance = new Database();
        }
        return instance;
    }
};

Database* Database::instance = null;
```

Notes

```
Database * db1 = Database::getInstance();
Database * db2 = Database::getInstance();
```

Both points to the same object.

(2) Factory Method Pattern: Provides an interface for creating objects, but lets the subclasses decide which object to create.

```
class Car{};
class Bike{};
int main(){  Car *c = new Car(); }
```

Code becomes tightly coupled

With Factory:

```
class Vehicle{
public: virtual void drive()=0;
};
class Car : public Vehicle{
public:
    void drive() override {};
};
class Bike : public Vehicle{
public:
    void drive() override {};
};
```

Notes

* Factory class:

```
class VehicleFactory {
public:
    static Vehicle* createVehicle(string type){
        if(type == "car") {
            return new Car();
        }
        else if(type == "bike") {
            return new Bike();
        }
        return null;
    }
};
```

Loose coupling
Easy to extend

```
Vehicle *v = VehicleFactory :: createVehicle("car");
v -> drive();
```

* Abstract Factory Pattern:

Provides an interface for creating families of related objects.

Factory creates multiple related objects.

E.x. GUI System

Windows:

WindowsButton, WindowsCheckbox

Mac:

MacButton, MacCheckbox

30 Sunday

Notes

* Builder Pattern:

Used to create complex objects step by step

When creating complex products (like a custom-built car), you don't want to handle it in onego.
The Builder Pattern lets you break down the creation process into steps.
E.x. Building a luxury car with step-by-step features.

---

* Prototype Pattern:
Creating new objects by copying existing objects.
Instead of creating from scratch → clone existing.

```
class shape {
    virtual shape* clone() = 0;
};
class circle : public shape {
    int r;
    circle(int r) { this.r = r; }
    shape* clone() override {
        return new circle(r);
    }
};
int main() {
    circle c1(10);
    circle *c2 = (circle*) c1.clone();
```

Notes

* Fast object creation
* Useful when object creation is expensive.

* Abstract Factory with example:

Factory method ⟶ Creates one product
E.x. createVehicle() ⟶ car or Bike

Abstract Factory ⟶ Create a family of related product.
E.x. createCar() & createBike() for different brands like Tesla, Toyota.

(1) Abstract Products:

```
class Car { virtual void drive() = 0; }
```

(2) Concrete Product ⟶ Tesla

```
class Tesla : public Car {
    void drive() override { };
};
```

(3) concreate Product ⟶ Toyota

```
class Toyota : public Car {
    void drive() override { };
};
```

(4) Abstract Factory

```
class VehicleFactory {
    virtual Car* createCar() = 0;
};
```

(5) Concrete Factories: Tesla car is created by tesla factory

```
class TeslaFactory : public VehicleFactory
{   Car* createCar() override {
        return new Tesla();
    }; }
```

Notes

* Toyota car is created by toyota factory:

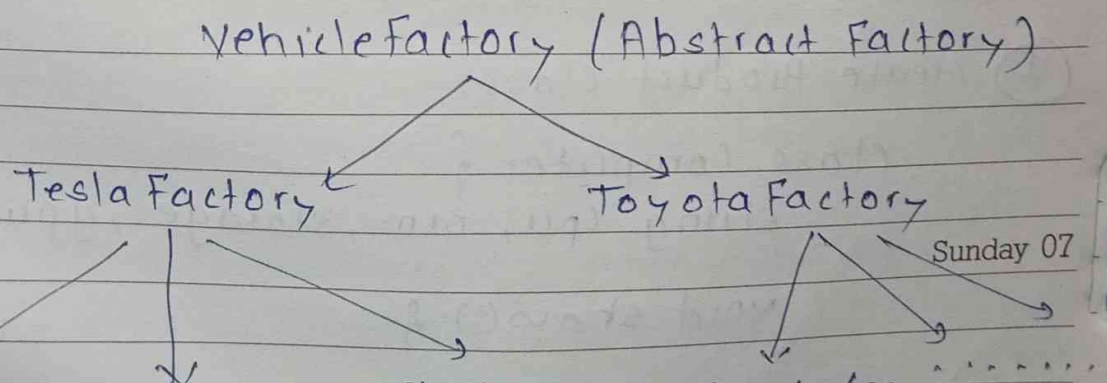
```
class ToyotaFactory : public VehicleFactory {
    car * createCar() override {
        return new ToyotaCar();
    }
};
```

Each factory creates a family of related products.

(6) Usage:

```
int main() {
    VehicleFactory * factory = new TeslaFactory();
    Car * c = factory -> createCar();
    c -> drive();  ==> Tesla car

    VehicleFactory * f2 = new ToyotaFactory();
    Car * c2 = f2 -> createCar();
    c2 -> drive();  ==> Toyota car
};
```



Notes

Sunday 07

AUGUST

* Builder Pattern with example: Used to construct complex objects step by step.

* Problem without builder pattern (Constructor Explosion)

```
class Computer {
    string CPU, RAM, storage, GPU;
    Computer (string c, string r, . . . . . ) {
            this.CPU = c;
            this.RAM = r;
                :
                :
    }
};
int main() { Computer c("Intel", "16GB",
             "1TB", "NVDIA"); }
```

what if any of these fields are optional
We would need too many constructors

* Builder Pattern:

(1) Create Product class:

```
Class Computer {
    string cpu, ram, storage, gpu;
    void show() {

};  }
```

This is the final object.

Notes

(2) Create a Builder class:

```
class ComputerBuilder {
private: Computer *computer;
public:
    ComputerBuilder() {
        computer = new Computer();
    }
ComputerBuilder &setCPU(string CPU) {
    computer -> CPU = CPU;
    return *this;
}
ComputerBuilder &setRAM(string RAM) {
    computer -> RAM = RAM;
    return *this;
}
```

returning the same builder object

```
Computer *build() {
    return computer;
};
}
```


Notes

10 July Wednesday 2024

192-174

Week 28

(3) Use Builder for creating object:

```
int main() {
    computer *pc = computerBuilder()
                    .setCPU("Intel i9")
                    .setRAM("16GB")
                    .setStorage("1TB SSD")
                    .setGPU("NVIDIA")
                    .build();

    pc->show();

    computer *pc2 = computerBuilder()
                    .setCPU("Intel i8")
                    .setGPU("512TB")
                    .build();

    pc2->show();
```

memory chaining

* Behavioral design patterns:

193-173

Focus on how objects communicate with each other & how responsibilities are distributed between objects.

(1) Strategy Pattern: Allows us to change the behaviour of an object dynamically without changing its code.

E.x. In an e-commerce application, user can pay using: credit card, UPI, PayPal.
So instead of writing multiple if-else or switch statements, we can create sperate strategy classes.

```
class PaymentStrategy {
        virtual void pay(int amount) = 0;
};
```

* ~~Concerete strategies~~:

```
class CreditCardPayment: public PaymentStrategy
{
        void pay(int amount) override { }
};
class UPIPayment: public PaymentStrategy {
        void pay(int amount) override { }
};
```

Notes

AUGUST

12 Friday 2024

194-172 Week 28

* context class

```
class PaymentContext {
        PaymentStrategy *strategy;
    void setStrategy (PaymentStrategy *s) {
        strategy = s;
    }
    void payAmount (int amount) {
        strategy → pay (amount);
    }
};
int main() {
    PaymentStrategy p;
    PaymentContext c;

    CreditPayment cc;
    UPIPayment u;

    c.setStrategy (&cc);
    c.payAmount (1000);

    c.setStrategy (&u);
    c.payAmount (2000);
```

E.x.
Google map provides different route strategies:
* car route strategy
* walking route strategy.
* Bike route strategy.
User selects strategy at runtime.

Notes

(2) Observer Design Pattern: Defines one-to-many dependency between objects so that when one object changes its state, all its dependent objects (observers) are automatically notified and updated.

E.x. YouTube channel ⟶ subject
subscribers ⟶ observers
When channel uploads a new video ⟶
All subscribers get notification.

(3) Iterator Pattern: Iterator allows you to travese (loop through) a collection without knowing how the collection is implemented internally.

E.x. TV remote.
TV ⟶ collection of channels. Remove ⟶ Iterator
Pressing next/previous ⟶ Iterator moves through channels.
You don't know how channels are stored internally.

Without iterator user need to know internal structure.

E.x. Iterator for Book Collection

Sunday 14

AUGUST

Notes

```
class Iterator {
    public: virtual bool hasNext() = 0;
            virtual bool next() = 0;
};
```

15 July Monday 2024

197-169

Week 29

(2) Concrete Iterator:

```
class BookIterator: public Iterator {
    vector<string> book.
    int index;

BookIterator (vector<string> b) {
        books = b ;  index = 0;
}
bool hasNext() override {
        return index < book.size();
}
string next() override {
        return book[index++];
};
}
```

(3) Collection:

```
class BookCollection {
    vector <string> books;
public
    void add (string book) {
        books.push_back(book);
}
  BookIterator createIterator() {
        return BookIterator (books);
}
};
```

Notes

(4) Main Function:

```
int main() {
    BookCollection c;
    c.addBook("C++");
    c.addBook("Java");

    BookIterator it = c.createIterator();
    while (it.hasNext()) {
        cout << it.next();
    }
}
```

* Command Design pattern: Turns a request into an object so it can be executed later, stored or undone.

E.x. When you press a button on a remote:

Remote button ⟶ command

TV ⟶ Receiver

Turn on / off ⟶ Action

The remote does not know how TV works internally. It just sends a command.

* Mediator Design Pattern: Mediator pattern defines an object called a mediator that controls communication between multiple objects.

Notes

Instead of objects communicating directly, they communicate through the mediator.

17 July Wednesday 2024

199-167 Week 29

E.x. Air Traffic control
Planes do not communicate directly with each other.

* State Design Pattern: state Pattern lets an object behave differently depending on its current state.

E.x. Traffic Light → same traffic lights, but behaviour changes based on state.

(1) class TrafficLightState{
    virtual void handle() = 0;
};

(2) concerete states:

class RedState : public TrafficLightState {
    void handle() overide {
        cout << "Red Light → stop";
    }
};

(3) context class

class TrafficLight {
    TrafficLightState * state;
public:
    void setState (TrafficLightState *s){
        state = s;
    }

Notes

```
    void showSignal() {
        state → handle();
    }
};
int main() {
    TrafficLight light;
    RedState r; GreenState g; YellowState y;

    light.setState(&r);
    light.showSignal();
    light.setState(&g);
    .
    .
    .
}
```

- Other examples:
  - ATM Machine → Idle, Processing, Out of Cash
  - Media Player → Playing, Paused, Stopped.

---

* Template Design pattern: Defines the skeleton of an algorithm in a base class, but allows subclasses to override specific steps without changing the overall structure.

E.x. Making tea & coffee
(1) Boil water (2) Add main ingredient
(3) Pour into cups (4) Add extras (sugar/Milk)
Structure is same, but some step differs.

Notes

AUGUST

(1) class Beverage {
public: void prepare() {
boilWater(); addMainIngredient();
pourInCup(); addExtras();
}
void boilWater() { };
void pourInCup() { };
virtual void addMainIngredient()=0;
virtual void addExtras()=0;
};

(2) concrete Tea class:
class Tea: public Beverage{
void addmainIngredient() override { }
void addExtras() override { }
};

(3) Concrete Coffee class:
class Coffee: public Beverage {
:
};

(4) Main Function:
int main() {
Tea t; Coffee c;
t.prepare();
c.prepare();
}

Notes

* Chain of Responsibility Pattern: Allows a request to pass through a chain of handlers, where each handler decides whether to process the request or pass it to the next handler.

The request moves through multiple objects until one object handles it.

E.x. Customer support System

When you raise a complaint

1. Level 1 support ⟶ handles basic issues
2. Level 2 ⟶ handles technical issues
3. Manager ⟶ handles critical issues

if level 1 cannot handle ⟶ passes to level 2 ⟶ then Manager.

This forms a chain.

---

* Visitor design pattern: Adding new operations to a group of related objects without modifying their classes.

E.x. Shopping Cart

Elements: Book, Electronics

operations: calculate price, Apply tax, generate invoice.

Instead of adding tax logic inside Book/Electronic, we create a Visitor.

Step 1: Visitor Interface

```
class Book;
class Electronic;

class Visitor {
public:   virtual void visit (Book * book) = 0;
          virtual void visit (Electronics* electronics) = 0;
};
class Item {
public:   virtual void accept (Visitor *visitor) = 0;
};
class Book : public Item {
        int price;
        Book (int p) { price = p; }
        void accept (Visitor *visitor) override {
                visitor -> visit (this);
        }
};

class PriceVisitor : public Visitor {
public:  void visit (Book *book) override { };
};
int main () {
        Book book(500); PriceVisitor visitor;
        book.accept (& visitor);
}
```

Notes

AUGUST 2024

S	M	T	W	T	F	S
				1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31

July
Tuesday
2024

23

Week 30

205-161

* Memento Design Pattern: Memento is used to implement undo/rollback functionality.

E.x. Text editor

when you type something:

* You save the current state
* If you pressed undo, it restores the previous state.

Structural Design Patterns

* Creational Design Patterns: How to create objects
* Behavior Design Patterns: How objects communicate
* Structural Design Patterns: How do we organize classes & objects so the system remains flexible and scalable.

(1) Adapter Design Pattern: Acts as a translator between two classes.

It allows two incompatible interfaces to work together.

E.x. (1) Your laptop charger has an Indian plug.
(2) The wall socket in U.S is difficult.
(3) You use a plug adapter.
(4) The adapter's doesn't change the charger - it just make it compatible.

E.x. You are building a payment system.
Your system expects:

```
class PaymentProcessor {
    virtual void pay(int amount) = 0;
};
```

But the payment gateway provided is a third party support.

Notes

```
class RazonPayGateway {
void makePayment(double amount) {
}
};
```

Problem: Your system calls ⟶ pay()
Razorpay has ⟶ makePayment()
Interfaces don't match

* Create Adapter class

```
class RazorPayAdapter : public PaymentProcessor {
private:
    RazorPayGateway *gateway;
public:
    RazorPayAdapter(RazorPayGateway *g)
        { gateway = g; }

    void pay(int amount) override {
        gateway ⟶ makePayment(amount);
    }
};
int main() {
    RazorPayGateway *r = new RazorPayGateway();
    PaymentProcessor *p = new RazorPayAdapter(r);
    p ⟶ pay(1000);
}
```

* Client talks to PaymentProcessor
* Adapter translates call.
* Actual work done by RazorPayGateway.

AUGUST

Notes

(2) Composite Design Pattern : Lets you treat individual objects and groups of objects uniformly. Mainly used to represent tree structures.

E.x. company Organization

Imagine a company :

* Developer (leaf) * Manager (composite)
* Director (composite)

We want to calculate salary.

```
class Employee {
public: virtual void showDetails() = 0;
        virtual int getSalary() = 0;
};
class Developer : public Employee {
    int salary;
    string name;
public:
    Developer (string s, int n) : name(s), salary(s)
    {
    void showDetails() override { }
    int getSalary() override {
        return salary;
    }
};
class Manager : public Employee {
    string name;
    int salary;
    vector<Employee*> subordinates;
```

```
public: Manager(string s, int n) : name(s), salary(n)
        {                                              }
void add(Employee *emp) {
    subordinates.push_back(emp);
}
void showDetails() override{
    cout << "Manger": . . . . - . ;
    for(auto emp : subordinates) {
        emp ⟶ showDetails();
    }          }
}
int getSalary override{
    int total = salary;
    for(auto emp: subordinates) {
        total += emp ⟶ getSalary();
    }
    return total;
}
};
int main() {
    Developer *d1 = new Developer("A", "20000");
    Developer *d2 = new Developer("B", 30000);
    Manager *m1 = new Manager("C", 200000);

    m1 ⟶ add(d1);
    m1 ⟶ add(d2);
    cout << "Total salary": m1 ⟶ getSalary();
}
```

Notes

AUGUST

Sunday 28

29 July Monday 2024

Week 31

211-155

* Facade Design Pattern: Providing a simple interface to a complex subsystem.

Instead of interacting with many classes, the client interacts with one unified interface (facade).

Hiding system complexity behind a simple interface.

E.x. Building a Home Theater System.

To watch a movie, the user must manually control: DVD Player, Projector, Sound System, Lights.

```
class DVDPlayer {
    void on() { }
    void play() { }
};
```

```
class Projector {
    void on() { }
};
```

```
class SoundSystem {
    void on() { }
};
```

Creating Facade:

```
class HomeTheaterFacade {
private: DVDPlayer d;
         Projector p;
         SoundSystem s;
public: void watchMovie() {
            d.on();
            d.play();
            s.on();
            p.on();
        }
};
```

```
int main() {
    HomeTheaterFacade
    t;
    t.watchMovie();
}
```

Notes

* Decorator Design Pattern: Allowing you to add new behavior to objects dynamically without modifying their code.

Attaching additional responsibilities to an object dynamically.

E.x. Ordering coffee:

(1) Basic coffee (2) coffee + milk

(3) coffee + milk + sugar (4) coffee + milk + sugar + caramel

Instead of creating many classes like, we decorate the coffee step by step.

Coffee ⟶ MilkDecorator ⟶ SugarDecorator ⟶ CaramelDecorator

```
class coffee {
    virtual int cost() = 0;
    virtual string description() = 0;
};
class SimpleCoffee : public coffee {
    int cost() override { return 50; }
    string description() override {
        return "simple coffee";
    }
};
```

* Base Decorator

```
class coffeeDecorator : public coffee {
public:     coffee * coffee;
            coffeeDecorator (coffee * c) {
                coffee = c;
};      }
```

```
class MilkDecorator : public CoffeeDecorator {
public: MilkDecorator(Coffee *c) : CoffeeDecorator(c) { }
    int cost() override {
        return coffee -> cost() + 20;
    }
    string description() override {
        return coffee -> description() + "Milk";
    }
};
```

```
int main() {
    Coffee *c = new SimpleCoffee();
    c = new MilkDecorator(c);
    cout << c -> description();
    cout << "Cost: " << c -> cost();
}
```

E.x. Logging systems:

* File logging
* network logging
* console logging.

Notes